
Group #43 Mid-term Report for Semantic Layout Acquisition from Movement

Zhongbo Yan

Information Networking Institute
Carnegie Mellon University
zhongboy@andrew.cmu.edu

Yiming Yin

Information Networking Institute
Carnegie Mellon University
yimingyi@andrew.cmu.edu

Abstract

We present *Room-SLAM*, a deep learning system that infers 3D semantic indoor layouts from movement traces captured by low-cost sensors like mmWave radar, eliminating the need for expensive LiDAR or privacy-invasive cameras. Formulated as a sequence-to-set prediction problem, our DETR-inspired architecture uses interchangeable Bidirectional LSTM or Transformer encoders to process kinematic-augmented trajectories, followed by a query-based decoder that predicts 3D bounding boxes and height-stratified object classes via cross-attention. Training employs Hungarian matching with a multi-task weighted loss and augmentation strategies including rotations, translation, scaling, and temporal warping. Evaluated on metrics including mIoU, precision/recall, and classification accuracy, our approach aims to prove that rich semantic maps can be reconstructed from simple motion data, enabling privacy-preserving smart environments without specialized hardware. We compare against a grid-based baseline to validate the effectiveness of learned representations over heuristic methods.

1 Introduction

Generating semantic maps of indoor environments is crucial for applications in robotics and smart-home automation, but current methods often rely on expensive sensors like LiDAR or privacy-invasive cameras. This project, *Room-SLAM*, explores a novel, low-cost, and privacy-preserving approach to infer a room’s semantic layout—including walls and obstacles at various heights—purely from 3D movement traces captured by affordable sensors like mmWave radar.

We frame this as a **sequence-to-set learning problem** inspired by DETR [1]. Given an input trace sequence $\mathbf{T} = \{(x_i, y_i, z_i, t_i)\}_{i=1}^N$ enhanced with kinematic features (velocity $\mathbf{v}_i$, acceleration $\mathbf{a}_i$, speed s_i), the model predicts a set of Q colliders $\mathcal{C} = \{(b_j, c_j)\}_{j=1}^Q$, where each collider consists of:

- A 3D bounding box $b_j = (c_x, c_y, c_z, w, h, d) \in \mathbb{R}^6$ representing center coordinates and dimensions (width, height, depth)
- A class label $c_j \in \{\text{BLOCK}, \text{LOW}, \text{MID}\}$, where BLOCK refers to obstacles blocking access (walls, high cupboards, fridges, gas cookers), LOW refers to low, sittable furniture (chairs, sofas, beds), and MID refers to mid-height surfaces typically used for placing objects (tables, counters)

2 Related Work

This project addresses a novel problem: inferring 3D semantic layouts from passive movement traces without visual or LiDAR sensors. We review relevant literature across three key areas: movement-to-scene reconstruction, set prediction architectures, and data collection methods.

2.1 Traditional SLAM and Semantic Mapping

Visual and LiDAR SLAM. Classical SLAM algorithms [2], [3] rely on active sensing—cameras (ORB-SLAM [4], PTAM [5]) or LiDAR (Cartographer [6], LOAM [7])—to simultaneously localize and map environments. While effective, these methods require expensive hardware (\$1000+ for LiDAR) and raise privacy concerns in residential settings. Recent semantic SLAM approaches [8], [9] augment geometric maps with object-level understanding, but still depend on RGB-D or LiDAR inputs.

2.2 Movement-to-Scene Reconstruction

Motion Pattern Analysis. Prior work demonstrates that human motion patterns encode semantic information about environments. Wang et al. [10] use WiFi CSI for activity recognition, while Yao et al. [11] demonstrate device-free localization. However, these methods do not reconstruct spatial layouts. Our key hypothesis—that movement is *constrained and guided* by physical objects—extends this line of research to scene-level understanding.

Privacy-Preserving Sensing. Millimeter-wave radar has emerged as a privacy-preserving alternative for indoor sensing [12]. Our work leverages the insight that affordable mmWave sensors can provide movement traces sufficient for inferring room layouts, enabling semantic scene reconstruction without visual data.

3 Dataset and Data Collection Application

To unify synthetic simulation and real-world sensing, we developed a custom application called **RoomSLAM**. Built on Unity3D, RoomSLAM serves as both a simulator and a data aggregator: it procedurally generates or imports room environments, assigns collider boxes to objects, and records trajectories from virtual agents. At the same time, RoomSLAM integrates with an iOS companion app (based on ARKit) that streams real-world user trajectories into Unity via UDP. This dual capability allows us to collect agent-generated traces and real-world human traces in a consistent format, ensuring that both data modalities are aligned with the same collider annotations and can be directly used for training and evaluation, the client and App is shown in Figure 1.

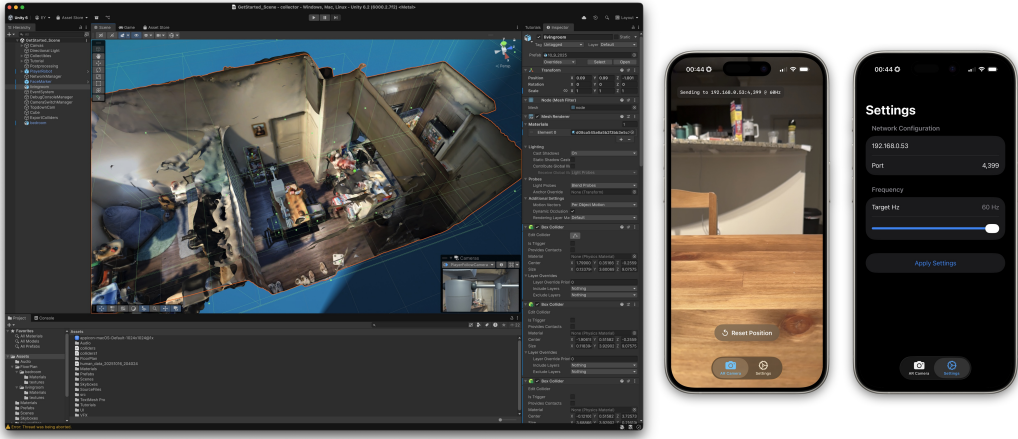


Figure 1: RoomSLAM build on macOS using Unity 3D and companion App using ARKit on iOS.

3.1 Unity3D Simulation

To provide controlled yet realistic environments, we implemented a Unity-based simulation framework with two manually modeled indoor scenes: a living room and a bedroom. Using LiDAR scans as a reference, we reconstructed these rooms with accurate geometry and collider assignments for walls, furniture, and objects. Within Unity, a mobile robot agent was deployed to navigate freely across the

space. The agent followed collision-aware trajectories with smooth turning dynamics and variable speed, mimicking natural human-like motion. Trace data were sampled at 60Hz, recording agent pose (x, y, z, t) together with synchronized ground-truth colliders. Each run produced JSON files containing both trajectory traces and associated 3D bounding boxes for obstacles. Unity’s recorder was used to log these data directly to disk, ensuring consistency and reproducibility across sessions.

In Dataset 1 (living room), we collected four agent-driven trajectories comprising **80,314 frames** in total, alongside three human-controlled trajectories with **22,582 frames**. In Dataset 2 (bedroom), one human-driven trajectory was recorded with **3,588 frames**. All collider annotations were automatically exported by Unity during collection, ensuring precise object-level ground truth.

3.2 ARKit Real-World Collection

To complement the synthetic data, we developed an iOS application (*SLAM World Sender*) based on Apple’s ARKit. The app uses ARKit’s visual-inertial odometry to estimate the user’s 6DOF pose in real time at 60Hz. Users calibrate their initial position relative to the Unity environment, after which trajectories are streamed via UDP directly into the Unity simulator. This enables *real-time world replay*, where human motion in physical rooms drives agents inside the simulated environment. The app simultaneously logs raw traces (x, y, z, t) , while ground-truth furniture and room structures are provided by Unity’s collider assignments, anchored to the calibrated origin.

This dual setup—synthetic Unity simulation with ground-truth colliders and real-world ARKit-driven replay—ensures consistency across domains while enabling direct comparison between simulated and real-world traces. The combination of controlled synthetic data and naturalistic real-world motion provides a robust basis for training and evaluating motion-to-scene reconstruction models.

4 Baseline Method

To establish a reference point for evaluation, we implement a **grid-based heuristic baseline** that reconstructs obstacle colliders through geometric reasoning rather than learned representations (see Algorithm 1).

4.1 Hyperparameters

The baseline has four key hyperparameters optimized via grid search on validation data:

- **Grid size** $g \in \{0.05, 0.1, 0.15, 0.2\}$ m: controls discretization resolution
- **Margin** $m \in \{0.2, 0.3, 0.4\}$ m: defines navigable buffer around trace
- **Max distance** $d_{\max} \in \{1.5, 2.0, 2.5\}$ m: limits obstacle detection range
- **Min block size** $s_{\min} \in \{0.3, 0.5, 0.8, 1.0, 1.5\}$ m: filters small spurious blocks

The grid search evaluates all combinations (180 total) and selects the configuration that maximizes mIoU on the validated traces held.

4.2 Baseline Performance

We evaluated the grid-based baseline through exhaustive hyperparameter search across two datasets: Dataset1 human_data_20251016_204024 and Dataset2 human_data_20251015_181004. Each dataset was tested with 180 different parameter combinations spanning grid sizes, minimum block sizes, margins, and maximum distances. The best configurations for each dataset are presented below.

4.2.1 Dataset1 Results

Table 1 shows the top 3 configurations for Dataset1. The best performance achieves mIoU of 0.2124 with moderate grid size (0.15m) and large minimum block size (1.5m), suggesting that this environment benefits from consolidated obstacle representations rather than fine-grained segmentation.

Algorithm 1 Grid-based Heuristic Baseline

Require: Agent trajectory $\mathbf{T}$, grid resolution g , buffer margin m , maximum distance $d_{\max}$, minimum block size $s_{\min}$

Ensure: Set of obstacle colliders $\mathcal{C}$

- 1: **Grid Rasterization:** Discretize the 2D environment (XZ-plane) into a uniform grid of size g ; initialize all cells as unoccupied.
- 2: **Trace Buffering:** Construct a polyline from $\mathbf{T}$ and buffer it by m :

$$\mathcal{B} \leftarrow \text{buffer}(\text{LineString}(\mathbf{T}), m)$$

Mark all grid cells intersecting $\mathcal{B}$ as navigable.

- 3: **Distance Filtering:** For each grid cell (i, j) :
 - 4: **if** $\text{cell}_{i,j} \cap \mathcal{B} \neq \emptyset$ **then**
 - 5: $\text{occupied}(i, j) \leftarrow \text{True}$
 - 6: **else if** $\min_{\mathbf{p} \in \mathbf{T}} \|\text{cell}_{i,j} - \mathbf{p}\| > d_{\max}$ **then**
 - 7: $\text{occupied}(i, j) \leftarrow \text{True}$
 - 8: **else**
 - 9: $\text{occupied}(i, j) \leftarrow \text{False}$
 - 10: **end if**
 - 11: **Rectangle Merging:** For each unoccupied cell (i, j) :
 - 12: Find maximal width $w_{\max}$: scan right until hitting occupied/visited cell
 - 13: Find maximal height $h_{\max}$: expand down while all cells in width are free
 - 14: **if** $w_{\max} < s_{\min}/g$ **and** $h_{\max} < s_{\min}/g$ **then skip**
 - 15: Mark rectangle as visited, convert to world coordinates
 - 16: **if** width $\geq s_{\min}$ **or** height $\geq s_{\min}$ **then** add to rectangles
 - 17: **Collider Generation:** For each rectangle r :
 - 18: Create 3D box $b \leftarrow (r.x, y_{\text{center}}, r.z, r.w, y_{\text{size}}, r.h)$
 - 19: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(b, \text{BLOCK})\}$
 - 20: **return** $\mathcal{C}$
-

Table 1: Top 3 baseline configurations on Dataset1

Rank	g (m)	$s_{\min}$ (m)	m (m)	$d_{\max}$ (m)	mIoU	Prec.	Rec.	F1
1	0.15	1.5	0.3	2.0	0.2124	0.6638	0.6638	0.6638
2	0.20	1.5	0.3	2.0	0.2120	0.6628	0.6628	0.6628
3	0.15	1.0	0.3	2.0	0.2105	0.6575	0.6575	0.6575

4.2.2 Dataset2 Results

Dataset2 achieves significantly higher performance (Table 2), with best mIoU of 0.3147. Interestingly, this dataset favors fine-grained grid resolution (0.05m) with smaller minimum block sizes (0.3-0.5m), indicating denser obstacle distributions that benefit from detailed spatial discretization.

Table 2: Top 3 baseline configurations on Dataset2

Rank	g (m)	$s_{\min}$ (m)	m (m)	$d_{\max}$ (m)	mIoU	Prec.	Rec.	F1
1	0.05	0.5	0.3	2.0	0.3147	0.7644	0.7644	0.7644
2	0.05	0.3	0.3	2.0	0.3145	0.7644	0.7644	0.7644
3	0.20	0.5	0.3	2.0	0.3142	0.7633	0.7633	0.7633

4.2.3 Analysis and Limitations

Several key observations emerge from the baseline evaluation:

- **Dataset sensitivity:** Performance varies significantly between datasets (mIoU 0.21 vs 0.31), suggesting that optimal hyperparameters are environment-specific and do not generalize well.

- **Precision-recall balance:** All configurations show identical precision and recall values, indicating that the baseline method produces a fixed trade-off determined by the hyperparameters rather than adapting to local scene complexity.
- **Grid size trade-off:** Dataset1 prefers coarser grids (0.15-0.20m) while Dataset2 benefits from fine resolution (0.05m), reflecting differences in obstacle density and room layout.
- **Consistent parameters:** Both datasets converge to $m = 0.3\text{m}$ and $d_{\max} = 2.0\text{m}$ across all top configurations, suggesting these are robust choices for the trace buffering and detection range.

These results establish a competitive baseline for comparison with our learned approach. The baseline’s inability to generalize across environments and distinguish obstacle heights motivates the need for data-driven methods that can learn complex spatial patterns.

While computationally efficient ($<1\text{s}$ per trace), the baseline has fundamental limitations:

- **No height reasoning:** Cannot distinguish between BLOCK, LOW, and MID classes; all obstacles are uniformly labeled as BLOCK
- **Axis-aligned only:** Produces only rectangular colliders aligned with coordinate axes, unable to capture rotated or irregular obstacles
- **Trace-dependent:** Performance degrades when agent coverage is sparse or biased toward certain regions
- **No generalization:** Hyperparameters tuned for one environment may not transfer to rooms with different layouts or obstacle densities
- **Over-segmentation:** Tends to produce many small boxes rather than consolidated obstacles

These limitations motivate our learning-based approach, which can capture complex spatial patterns, multi-class semantics, and generalize across diverse environments from training data.

5 Solution

5.1 Model Architecture

Our architecture to solving this problem using time-series models consists of three components: encoder, decoder, and prediction heads (Figure 2).

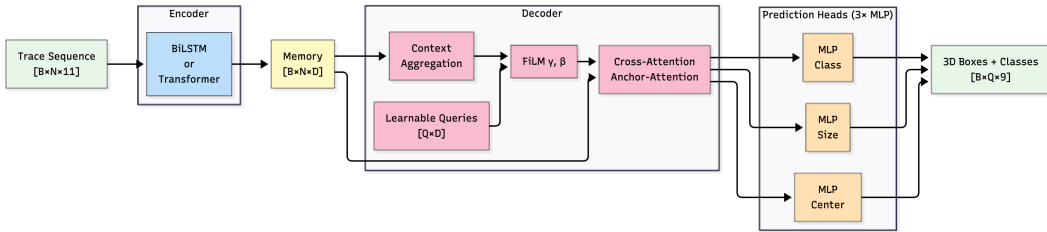


Figure 2: Overall model architecture for indoor semantic reconstruction. The pipeline consists of input trajectory encoding, grid-based reasoning, and collider prediction, culminating in obstacle geometry reconstruction.

Two encoder architectures are implemented: a **Bidirectional LSTM** for efficient processing and a **Transformer encoder** for capturing long-range dependencies. Both produce contextualized representations $\mathbf{M} \in \mathbb{R}^{N \times D}$ along with normalization statistics (μ, σ) for the prediction of relative coordinates.

The decoder employs Q **learnable object queries** $\{\mathbf{q}_j\}_{j=1}^Q$ that cross-attend to the encoded features. Through dot-product attention:

$$\text{Attn}(\mathbf{q}_j, \mathbf{M}) = \text{softmax} \left(\frac{\mathbf{q}_j \mathbf{M}^\top}{\sqrt{D}} \right) \quad (1)$$

each query identifies an anchor position and predicts relative offsets $\Delta \mathbf{c}_j$ and sizes s_j to produce the final bounding boxes and class predictions.

5.2 Training Objective

Training employs **Hungarian matching** to optimally assign predictions to ground truth, minimizing a multi-task weighted loss. The complete loss function consists of five components:

$$\mathcal{L} = \lambda_{\text{cls}} \mathcal{L}_{\text{CE}} + \lambda_{L_1} \mathcal{L}_{L_1} + \lambda_{\text{GloU}} \mathcal{L}_{\text{GloU}} + \lambda_{\text{cov}} \mathcal{L}_{\text{coverage}} + \lambda_{\text{avoid}} \mathcal{L}_{\text{avoidance}} \quad (2)$$

where:

- $\mathcal{L}_{\text{CE}}$ is cross-entropy for classification
- $\mathcal{L}_{L_1} = \|\mathbf{b}_{\text{pred}} - \mathbf{b}_{\text{gt}}\|_1$ measures box regression error
- $\mathcal{L}_{\text{GloU}} = 1 - \text{GloU}(\mathbf{b}_{\text{pred}}, \mathbf{b}_{\text{gt}})$ provides 3D IoU-based localization
- $\mathcal{L}_{\text{coverage}}$ penalizes predicted colliders that overlap with the agent’s trace, ensuring navigable space remains unoccluded:

$$\mathcal{L}_{\text{coverage}} = \sum_{j=1}^Q \sum_{i=1}^N \mathbb{I}[\mathbf{p}_i \in b_j] \quad (3)$$

where $\mathbf{p}_i = (x_i, z_i)$ is the 2D position of the i -th trace point

- $\mathcal{L}_{\text{avoidance}}$ encourages colliders to be placed near obstacle boundaries implied by the trace, computed as the negative minimum distance from predicted boxes to the trace buffer boundary

Aggressive data augmentation includes rotations ($\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$), collider dropout, sequence reversal, Gaussian noise, temporal cropping, and time-warping to improve generalization.

5.3 Evaluation Metrics

We evaluated the effectiveness of the indoor semantic reconstruction task by comparing predicted 3D collision boxes with ground-truth annotations. The following metrics are used:

- **Mean Intersection over Union (mIoU):** The average IoU across all classes, computed by rasterizing predicted and ground-truth colliders into 2D occupancy grids.
- **Precision, Recall, and F₁-score:** Detection quality is assessed at an IoU threshold of $\tau = 0.5$, capturing the trade-off between false positives and false negatives.
- **Classification Accuracy:** Per-class accuracy is reported for predictions that are successfully matched to ground-truth instances.

6 Future Experimental Plan

6.1 Single-Room Overfitting Analysis

First, we will analyze the model’s tendency to overfit. Although we have applied rotation-based augmentation (90° – 270°) to increase data diversity, we hypothesize that the model will still exhibit strong overfitting to individual house plans. We will investigate whether the network is establishing meaningful relationships between traces and layouts, or if it will tend to memorize the fixed topology of the training environment. This analysis will test our hypothesis that simple rotational augmentation preserves too much of the underlying spatial structure and will be insufficient to encourage genuine trace–collider reasoning. Moreover, our initial loss weighting strategy relies on a manually defined weight dictionary; we have not yet identified the optimal configuration of loss terms. Addressing this will require both more principled hyperparameter tuning and the collection of additional room layouts and trajectory traces.

A critical open question that we aim to answer is what the model will actually be learning. We will seek to determine if it can faithfully map traces to layouts, as intended, or if it will instead exploit spurious correlations tied to environment geometry.

6.2 Multi-Room Generalization

Finally, we will evaluate the model’s ability to generalize across multiple distinct room layouts. We will collect a dataset of five or more environments with diverse structural patterns, including open-plan spaces, corridor-like passages, and compartmentalized rooms.

Training will be conducted on $N - 1$ rooms while holding out one room for testing, rotating the held-out environment across folds. Evaluation will focus on cross-room mIoU and F1 scores to quantify detection accuracy, attention entropy to measure the degree of query–trace dependency, and prediction variance across trajectory subsets to assess robustness. This multi-room protocol will directly test the capacity of the model to move beyond layout-specific learning toward generalizable scene reconstruction.

References

- [1] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *European conference on computer vision*, Springer, 2020, pp. 213–229.
- [2] H. Durrant-Whyte and T. Bailey, “Simultaneous localization and mapping: Part i,” *IEEE robotics & automation magazine*, vol. 13, no. 2, pp. 99–110, 2006.
- [3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic robotics*. MIT press, 2005.
- [4] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, “ORB-SLAM: A versatile and accurate monocular SLAM system,” *IEEE transactions on robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.
- [5] G. Klein and D. Murray, “Parallel tracking and mapping for small AR workspaces,” in *2007 6th IEEE and ACM international symposium on mixed and augmented reality*, IEEE, 2007, pp. 225–234.
- [6] W. Hess, D. Kohler, H. Rapp, and D. Andor, “Real-time loop closure in 2d lidar slam,” in *2016 IEEE international conference on robotics and automation (ICRA)*, IEEE, 2016, pp. 1271–1278.
- [7] J. Zhang and S. Singh, “LOAM: Lidar odometry and mapping in real-time,” in *Robotics: Science and systems*, vol. 2, 2014, pp. 1–9.
- [8] J. McCormac, A. Handa, A. Davison, and S. Leutenegger, “SemanticFusion: Dense 3d semantic mapping with convolutional neural networks,” in *2017 IEEE International Conference on Robotics and automation (ICRA)*, IEEE, 2017, pp. 4628–4635.
- [9] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, “Kimera: An open-source library for real-time metric-semantic localization and mapping,” *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1689–1696, 2020.
- [10] H. Wang et al., “RT-Fall: A real-time and contactless fall detection system with commodity wifi devices,” 2, vol. 16, IEEE, 2017, pp. 511–526.
- [11] S. Yao, S. Hu, Y. Zhao, A. Zhang, and T. Abdelzaher, “Radio based device-free activity recognition with radio frequency interference,” in *Proceedings of the 14th international conference on information processing in sensor networks*, 2013, pp. 154–165.
- [12] A. Sengupta, F. Jin, R. Zhang, and S. Cao, “Mm-pose: Real-time human skeletal posture estimation using mmwave radars and cnns,” *IEEE Sensors Journal*, vol. 20, no. 17, pp. 10 032–10 044, 2020.